Exercise 1

Part 1

```python
#Calculate Prices of affordable products: 7% sales tax. remove any more than 25 bucks
ProductsList = {"Perfume": 20, "Body oil": 24, "Pajama Set": 25, "Slippers": 7, "Case": 15}

# user input
num_new_items = int(input("How many new items would you like to add? "))
for i in range(num_new_items):
    new_product_name = input("Enter the name of the new product: ")
    new_product_price = float(input(f"Enter the price of {new_product_name}: "))
    ProductsList[new_product_name] = new_product_price
#filter out
def FilterProducts_NoComprehension(products):
    affordable_products = {}
    for product, price in products.items(): #go through dictionary and get prices to get price with tax
        price_with_tax = price * 1.07
        if price_with_tax <= 25:
            affordable_products[product] = price_with_tax
    return affordable_products

affordable_products = FilterProducts_NoComprehension(ProductsList)
print(affordable_products)
```

```
How many new items would you like to add? 1
Enter the name of the new product: Socks
Enter the price of Socks: 7
{'Perfume': 21.400000000000002, 'Slippers': 7.49, 'Case': 16.05, 'Socks': 7.49}
```

Part 2

```python
# Calculate Prices
ProductsList = {"Perfume": 20, "Body oil": 24, "Pajama Set": 25, "Slippers": 7, "Case": 15}

# user input
num_new_items = int(input("How many new items would you like to add? "))
for i in range(num_new_items):
    new_product_name = input("Enter the name of the new product: ")
    new_product_price = float(input(f"Enter the price of {new_product_name}: "))
    ProductsList[new_product_name] = new_product_price

# Filter out and calculate price with tax using dictionary comprehension
affordable_products= {product: price * 1.07 for product, price in ProductsList.items() if price * 1.07 <= 25}

print(affordable_products)
```

```
How many new items would you like to add? 1
Enter the name of the new product: Sockss
Enter the price of Sockss: 7
{'Perfume': 21.400000000000002, 'Slippers': 7.49, 'Case': 16.05, 'Sockss': 7.49}
```

Exercise 2

Part 1

```python
#Digital Library Sysetm
Library = {}

def New_Book():
    num_new_books = int(input("Enter Amount of Books: "))
    for i in range(num_new_books):
        Book_Name = input("Enter Book Name: ")
        Book_Author = input("Enter Book Author: ")
        Library[Book_Name] = Book_Author
    return Library

print(New_Book())

#Add Book
Add_Book = input("Enter book name to add:")
Author = input("Enter author name:")
Library[Add_Book] = Author
print(Library)

#Get Author of book
Book = input("Enter book name to get their author:")
print(Library.get(Book))

#Update Author of book
Book_To_Update = input("Enter the book name to update the author:")
New_Author = input("Enter the new author name:")
Library.update({Book_To_Update: New_Author})
print(Library)

#Remove book
Book_To_Remove = input("Enter the book name to remove:")
Library.pop(Book_To_Remove)
print(Library)

#Display all books
print(Library)
```

```
Enter Amount of Books: 2
Enter Book Name: Dork Diaries
Enter Book Author: Rachel Russell
Enter Book Name: The Sour Grape
Enter Book Author: Jory John
{'Dork Diaries': 'Rachel Russell', 'The Sour Grape': 'Jory John'}
Enter book name to add:New Kid
Enter author name:Jerry Craft
{'Dork Diaries': 'Rachel Russell', 'The Sour Grape': 'Jory John', 'New Kid': 'Jerry Craft'}
Enter book name to get their author:Dork Diaries
Rachel Russell
Enter the book name to update the author:New Kid
Enter the new author name:Sandina Charles
{'Dork Diaries': 'Rachel Russell', 'The Sour Grape': 'Jory John', 'New Kid': 'Sandina Charles'}
Enter the book name to remove:New Kid
{'Dork Diaries': 'Rachel Russell', 'The Sour Grape': 'Jory John'}
{'Dork Diaries': 'Rachel Russell', 'The Sour Grape': 'Jory John'}
```

Part 2

```
Categories = set()
#add category
New_Category = input("Enter New Categories separated by a space: ")
Categories.update({category for category in New_Category.split()})

#remove category
Remove_Category = input("Enter Category to remove: ")
if Remove_Category in Categories:
  Categories.remove(Remove_Category)
else:
  print(f"{Remove_Category} is not in the set of categories.")

#check if category exists by comparing to a predefined set
Predefined_Categories = {'Fiction', 'Non-Fiction', 'Science', 'History'}

#Intersection between categories
Common_Categories = Categories.intersection(Predefined_Categories)
print(f"Common Categories: {Common_Categories}")

#Difference between predefined set and things missing from my set
Missing_Categories = Predefined_Categories.difference(Categories)
print(f"Missing Categories: {Missing_Categories}")
```

```
Enter New Categories separated by a space: Fiction History Science
Enter Category to remove: History
Common Categories: {'Science', 'Fiction'}
Missing Categories: {'History', 'Non-Fiction'}
```

Exercise 3

```python
shopping_list ={}
item_category = input("Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: ")
while item_category.lower() != "finish":
    item_name = input("Enter item name: ")
    if item_category not in shopping_list:
      shopping_list[item_category] = []
    shopping_list[item_category].append(item_name)
    item_category = input("Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: ")

print(shopping_list)

#find object, print its category if present else print its not there
Find_Item = input("Enter an item name to check its presence in the shopping list: ")
for category, items in shopping_list.items():
    if Find_Item in items:
        print(f"{Find_Item} is in the {category} category.")
        break
else:
    print(f"{Find_Item} is not in the shopping list.")
```

```
Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: Candy
Enter item name: M&M
Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: Cleaning Supplies
Enter item name: soap
Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: Snacks
Enter item name: Pretzels
Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: Candy
Enter item name: Kitkats
Enter the category for your item (e.g., Dairy, Snacks, Cleaning Supplies). Type 'FINISH' to stop: FINISH
{'Candy': ['M&M', 'Kitkats'], 'Cleaning Supplies': ['soap'], 'Snacks': ['Pretzels']}
Enter an item name to check its presence in the shopping list: M&M
M&M is in the Candy category.
```

Bonus Exercise

```
students_dict = {} #have dictionary to see each student and their grades
Students = int(input("How many students are there ")) #to know how many times the loop will go
for i in range(Students):
    name = input("Enter the name of student ") #key
    grade = input("Enter the grades seperated by space ") #vvalue
    grades = [int(i) for i in grade.split()] #each vaue turns into list with list comprehension
    students_dict[name] = grades #turn each value into the dictionary!!
print(students_dict) #print dictionary


def Find_Max_Grade_Person(students_dict): #findthe maximum score on the exams
  """
  all_grades = [i for i in students_dict.values()]
  best_grade = i for i in all_grades if max(all_grades)
  for name, grade in students_dict.items():
    if grade == best_grade: #didnt work, wonder why
      """

  for name, grades in students_dict.items(): #go through each pairing in the dicitonary
      for grade in grades: #go through the lsit of values in each pairing
        max_grade = 0 #initializaiton
        if grade > max_grade:
            max_grade = grade
            best_student = name
  return best_student

print(Find_Max_Grade_Person(students_dict))
```

```
How many students are there 2
Enter the name of student Leah
Enter the grades seperated by space 99 88 78
Enter the name of student Park
Enter the grades seperated by space 100 23 40
{'Leah': [99, 88, 78], 'Park': [100, 23, 40]}
Park
```

Python Program

Make a code that makes a dictionary to see each student and their grades using user input, then find the person with the highest SINGLE grade in their list of grades

e

```
def get_student_data():
    """
    Ask the user for student names and their grades.
    Returns a dictionary where each key is a student name and value is a list of grades.
    """
    students = {}
    try:
        n = int(input("How many students do you want to enter? "))
        if n <= 0:
            print("Number of students must be positive.")
            return {}
    except ValueError:
        print("Invalid input! Please enter a whole number.")
        return {}

    for i in range(n):
        name = input(f"\nEnter the name of student #{i + 1}: ").strip()
        if not name:
            print("Name cannot be empty! Skipping this student.")
            continue

        try:
            grades_input = input(f"Enter {name}'s grades (separated by spaces): ")
            grades = [float(g) for g in grades_input.split()]
            if not grades:
                print("No grades entered! Skipping this student.")
                continue
        except ValueError:
            print("Invalid grade format! Skipping this student.")
            continue

        students[name] = grades

    return students


def find_highest_single_grade(students):
    """
    Finds which student has the highest single grade among all.
    Returns a tuple (student_name, highest_grade).
    """
    if not students:
        return None, None
```

```
    top_student = None
    top_grade = -1

    for name, grades in students.items():
        highest_for_student = max(grades)
        if highest_for_student > top_grade:
            top_grade = highest_for_student
            top_student = name

    return top_student, top_grade


# --- Main Program ---

students = get_student_data()

if students:
    name, grade = find_highest_single_grade(students)
    if name:
        print(f"\nThe student with the highest single grade is {name} with a grade of {grade}.")
    else:
        print("\nNo valid student data found.")
else:
    print("\nNo students entered. Program ended.")
```

✅ Example Run:

```pgsql
How many students do you want to enter? 3

Enter the name of student #1: Alice
Enter Alice's grades (separated by spaces): 85 92 78

Enter the name of student #2: Ben
Enter Ben's grades (separated by spaces): 90 88 95

Enter the name of student #3: Cara
Enter Cara's grades (separated by spaces): 87 91

The student with the highest single grade is Ben with a grade of 95.0.
```

Reflection

On my first run of the program, it was similar to mine, which proved mine was efficient at the bare minimum.That was nice, but I wanted to see what else ChatGPT would do with the same prompt, and it gave me so much more. Mine definitely has more clarity, but ChatGPT put in more measures for error prevention. I prefer my style though I am aware it would probably be better to take measures for error. So AI definitely knows more than I did, so it does work more for the argument that learning to code doesn't matter, but, for example, the AI didnt print the full dictionary compared to what I wanted, which was to print the full dictionary too. Sure you could specify this to the AI, but in more complex problems it may not be able to read your mind and ultimately may cause mistakes that exponentiate.